

Grammars & Fractals

Programming and Music 2
<http://www.bjarni-gunnarsson.net>

Grammars

Syntax, (syntaxis in Greek) a term for an **arrangement**. The syntax dictates how words are arranged. It brings forth the relationship of **meaning within sentences**. Syntax deals with **combination** of words and their **structure**.

Grammar, (Grammatika in Greek) refers to the '**the art of writing**'.

“the study or use of the rules about how words change their form and combine with other words to express meaning”.

(Cambridge Dictionary)

Grammar refers to the set of rules in a given language (including syntax).

Grammars

Expressions in a given language are made up of a combination of units.

The **syntax** of the language allows for checking whether the given expression conforms to its rules (whether it is syntactically correct).

Sequences are correct if they conform to **syntactic rules** but this still does not guarantee that the phrases are meaningful.

Generative Grammars

Generative Grammars are a grammar type where ***formal rules*** are used to ***generate*** possible sentences in a language. Instead of analysing, the rules both generate and define the valid outcomes.

The generation of new sequences in a generative grammar occurs by means of ***rewriting rules***. Here, symbols on the left-hand side of an expression are rewritten by symbols on the right-hand side.

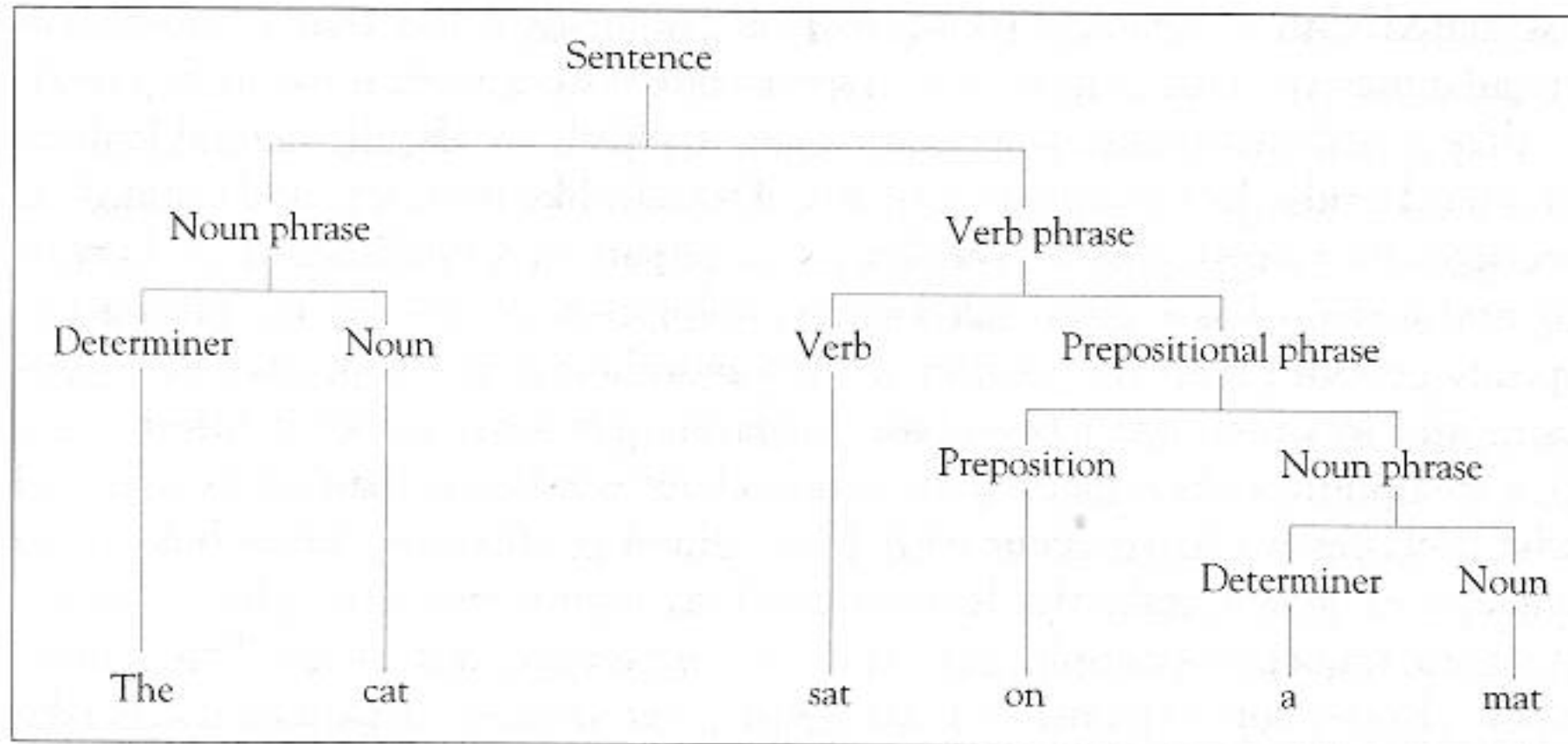
An important difference exists between symbols that can be rewritten (***non-terminal symbols***) and symbols that cannot (***terminal symbols***).

Generative Grammars

"Concepts that allow a hierarchical division of the musical material and work with the substitution of symbols can naturally be best formulated by a generative grammar." (Nierhaus)

The analysis form is ***transformational generative grammar*** is a generative grammar, usually of a natural language, that uses defined operations or transformations to generate new sentences from existing ones.

Generative Grammars



A possible phrase maker.

Generative Grammars

S (sentence)

NP (nominal phrase)

VP (verbal phrase)

V (verb)

V (verb)

PP (prepositional phrase)

AP (adjective phrase)

Adv (adverb)

Adv (adverb)

A (adjective)

P (preposition)

DET (article/determiner)

N (noun)

S	→	NP	VP		
VP	→	V	(NP)	(PP)	
AP	→	(Adv)	A	(PP)	
PP	→	P	NP		
NP	→	(DET)	(AP)	N	(PP)

N	→	man	girl	John
DET	→	a	the	
V	→	met	saw	
A	→	nice	good	quick
Adv	→	very	extremely	
P	→	in	for	to

(Nierhaus, Algorithmic Composition).

Generative Grammars

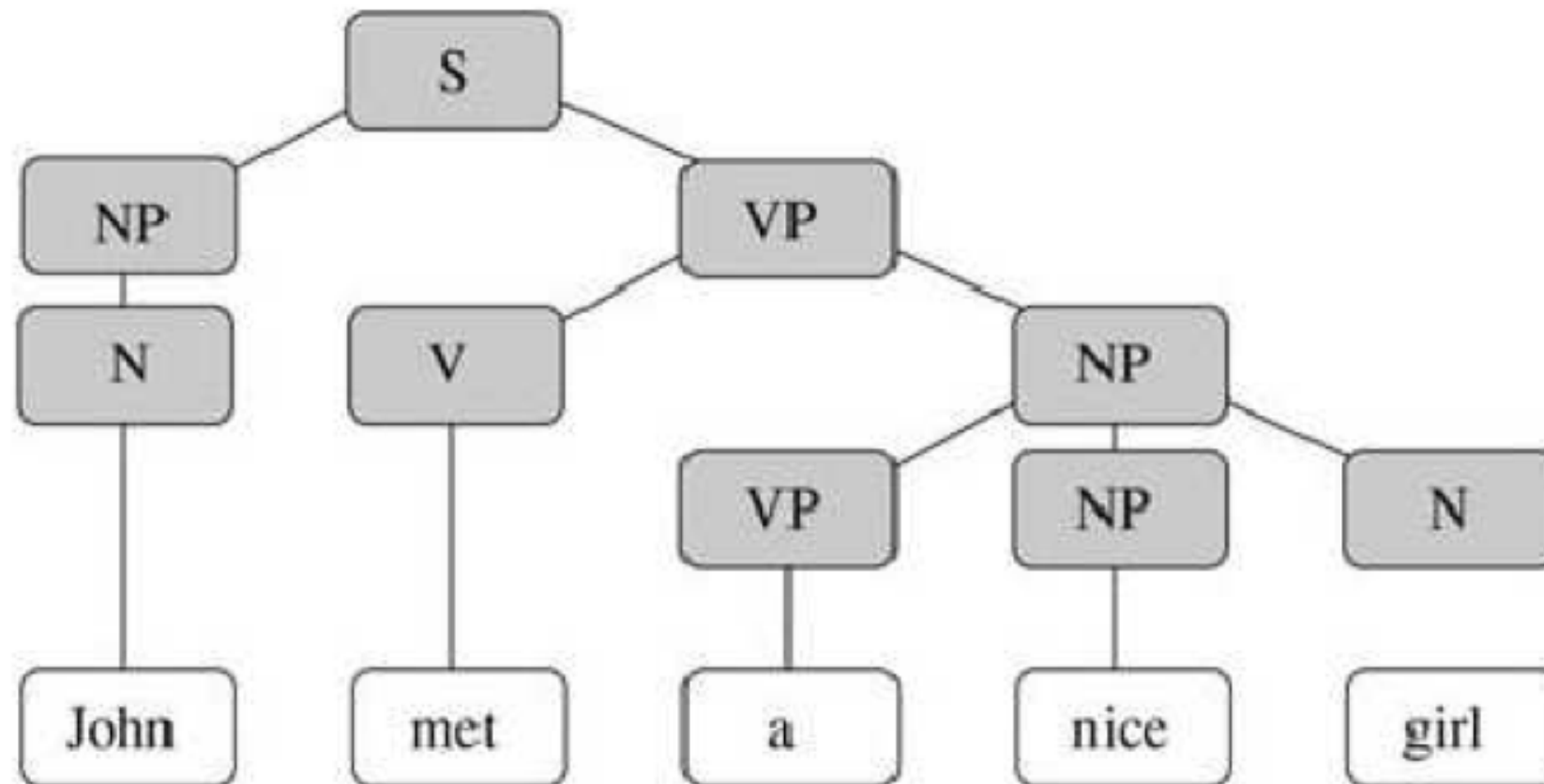


Fig. 4.2 Derivation of a PS-grammar.

(Nierhaus, Algorithmic Composition).

Deep Structure

Deep and *surface* structure.

The deep structure of a linguistic expression is a theoretical construct that seeks to unify several related structures.

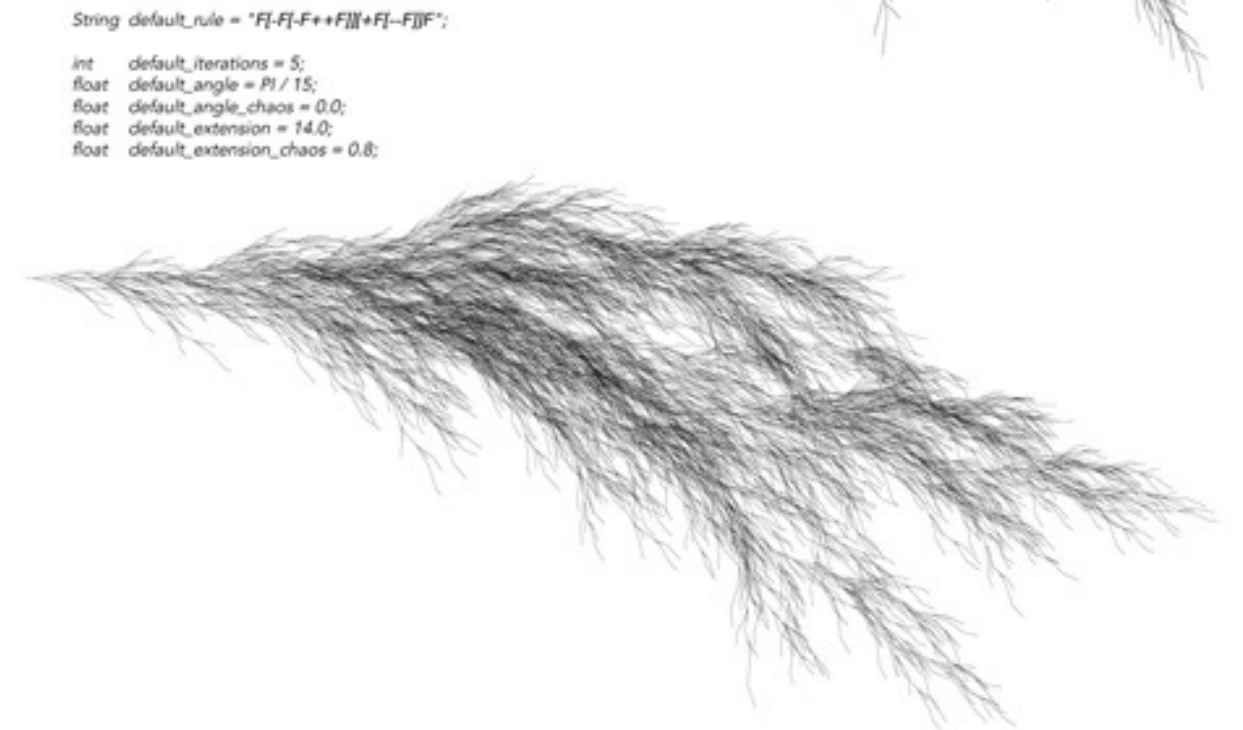
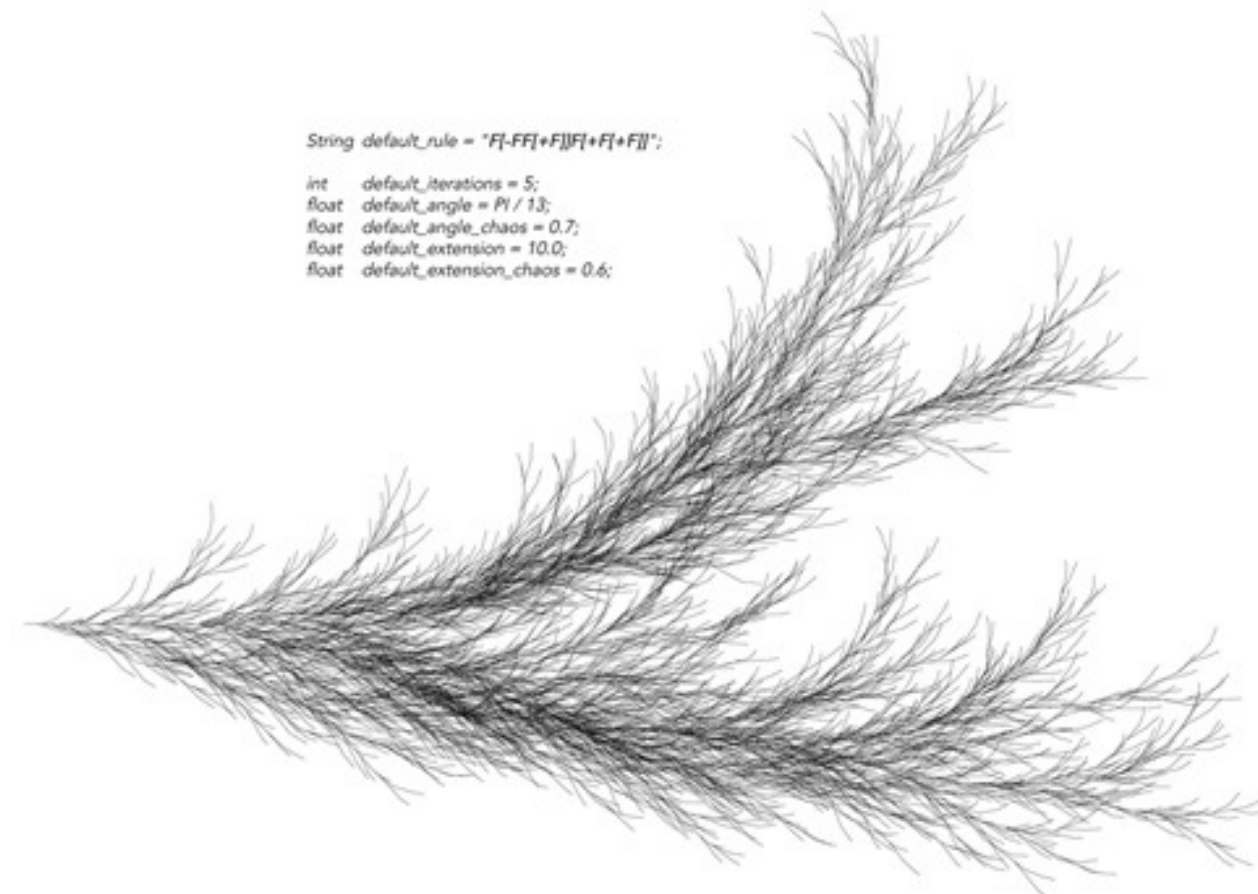
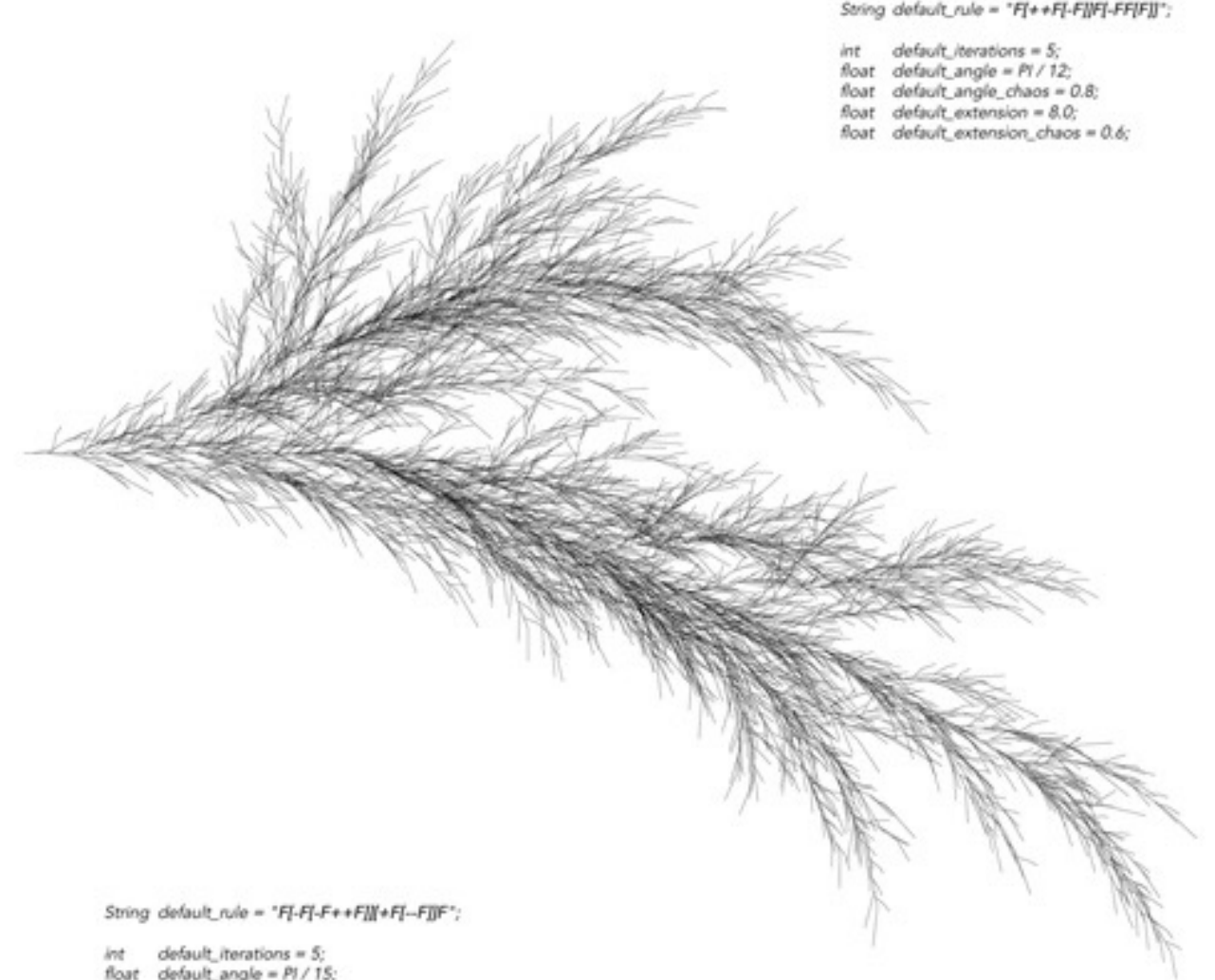
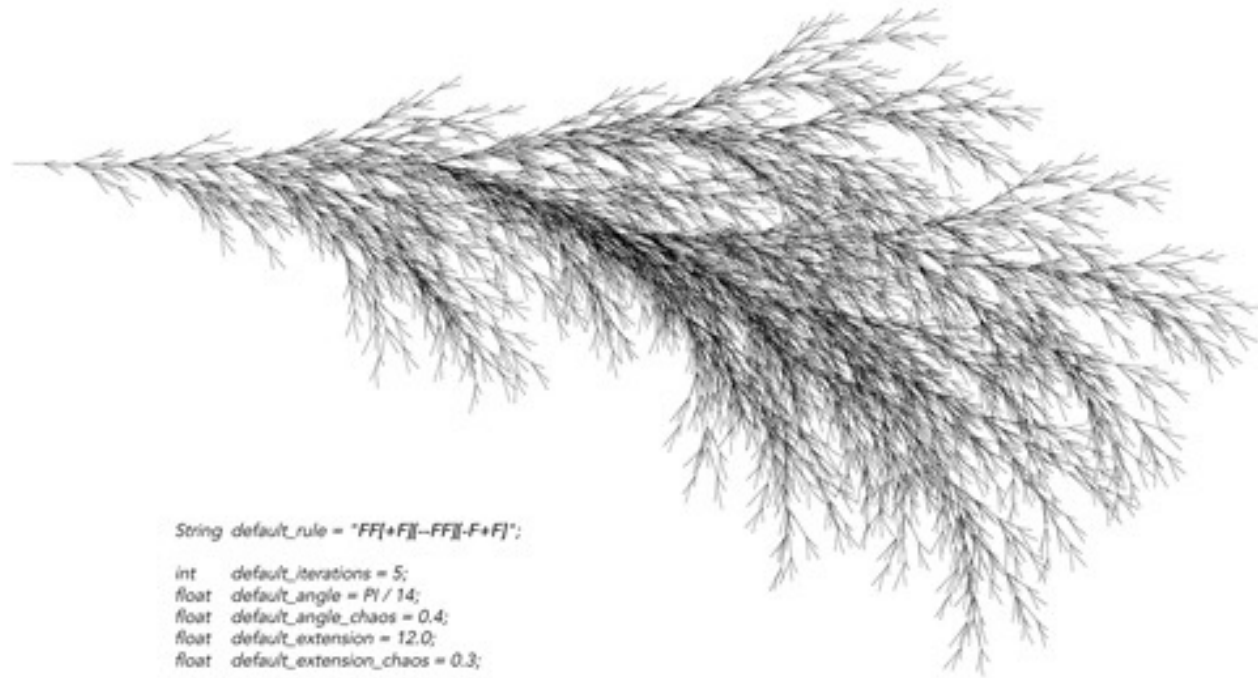
The difference between: *'Richard composed the music'* and *'The music was composed by Richard'* (active and passive form).

Some theories (Chomsky) suggest that these are the same surface structure of a common **deep structure**. Different approaches to convey the same meaning.

Could this apply for generative music algorithms ?

L-Systems

Lindenmayer Systems



Lindenmayer Systems

Self-similarity is a term used for structures where each of its part is a “*reduced- size copy of the whole.*”

Self-similar structures can be modeled by **Lindenmayer** systems (LS). These were originally developed as a formal language for the description of the growth process of plants and present a powerful tool for the generation of musical structure in the field of algorithmic composition.

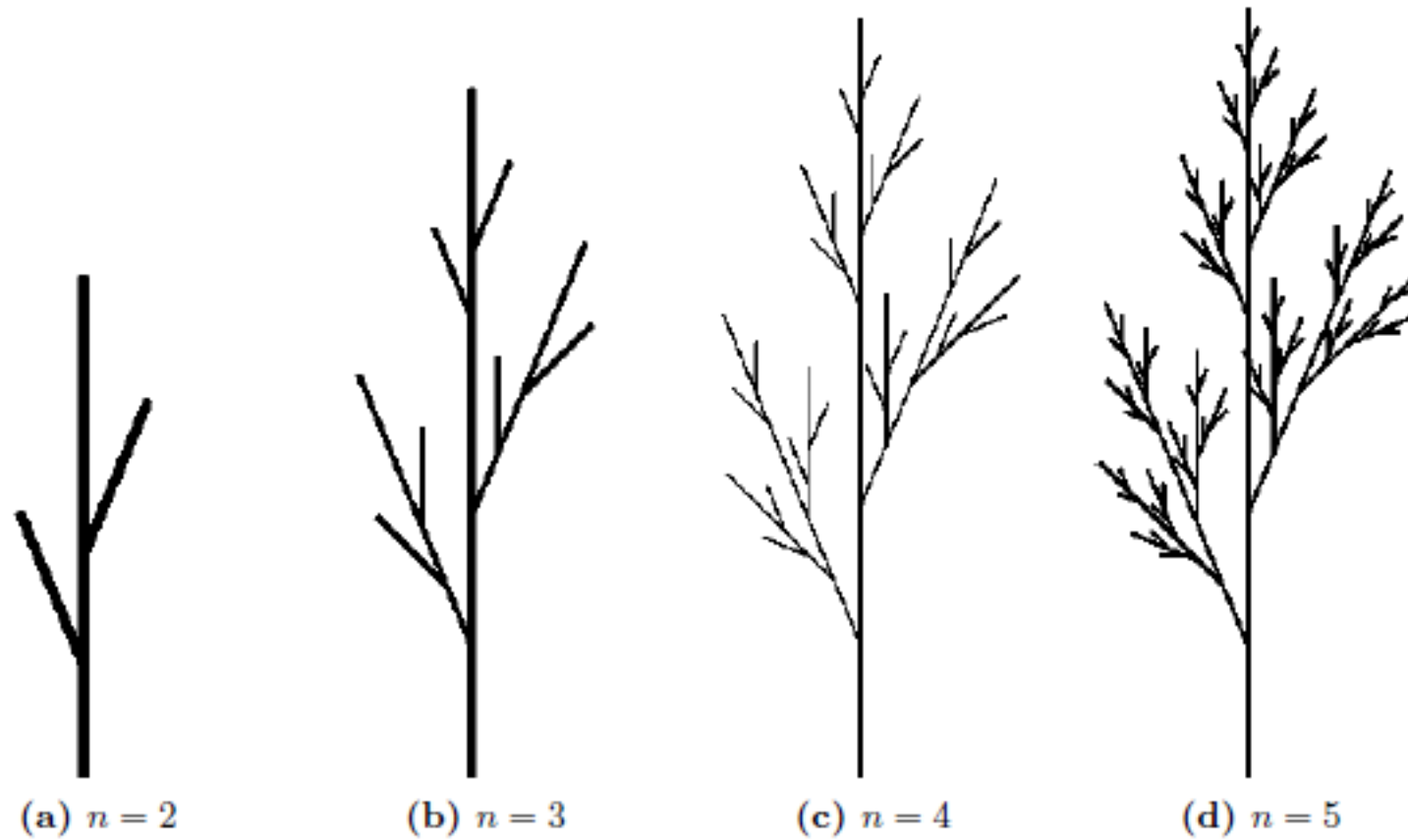
An L-system, is a **string rewriting** system that can be used to generate **fractals** and self-similar structures.

Lindenmayer Systems

Main elements of an L-System:

- **Alphabet.** Consists of the valid characters that can be included in the output. An example would be an alphabet of “ABC” where any valid sequence/sentence in the system can only include these three characters.
- **Axiom.** The sentence that describes the initial state of the system. With the alphabet “ABC” possible axioms are: “AA”, “CBAABC” or “B”.
- **Rules.** First applied to an axiom and then recursively to its output, generating new sentences. Each rule involves a “predecessor” and a “successor.” With the rule “ $A \rightarrow AB$ ”, whenever an “A” is found in a string, it is replaced with “AB.”

Lindenmayer Systems



Fractals

Fractals

The term comes from the latin word ***fractus*** (broken) and first used by ***Benoit Mandelbrot*** (1975).

“a rough or fragmented geometric shape that can be split into parts, each of which is (at least approximately) a reduced-size copy of the whole.”

Examples of fractals include *coastlines, stock markets* and *weather forecasts*. These often look the same at any scale making it hard to know what the window of operation is.

Fractals

Characterized by a ***fine structure*** at small scales, infinite ***details*** emerge the further one zooms into a fractal.

Fundamental concepts recurring in fractals are ***recursion levels***, ***children branches*** and ***complexity growth***.

Stochastic fractals are built out of ***probabilities*** and ***randomness***. Unlike the deterministic iterative self-similarity these are ***statistically*** self-similar.

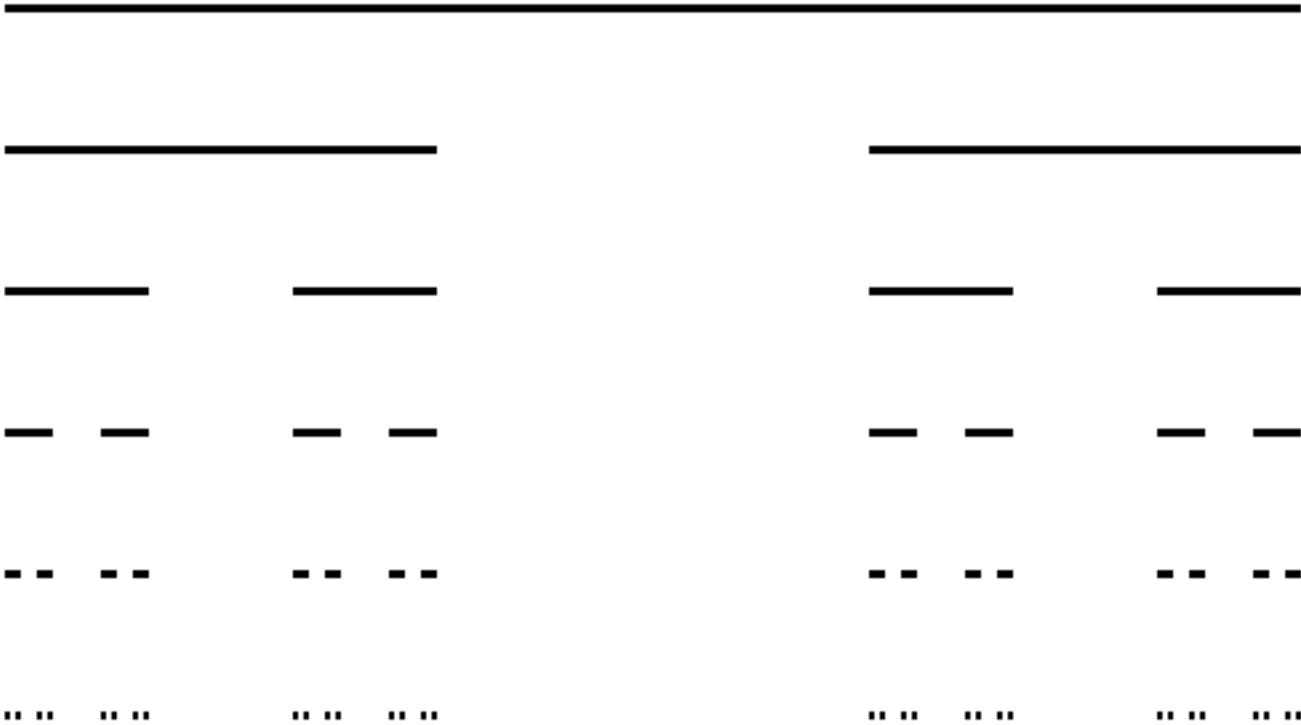
Recursion

A method of problem solving that where one ***breaks a problem down into smaller and smaller subproblems*** until you get to a small enough problem that it can be solved easily. Recursion usually involves a function ***calling itself***.

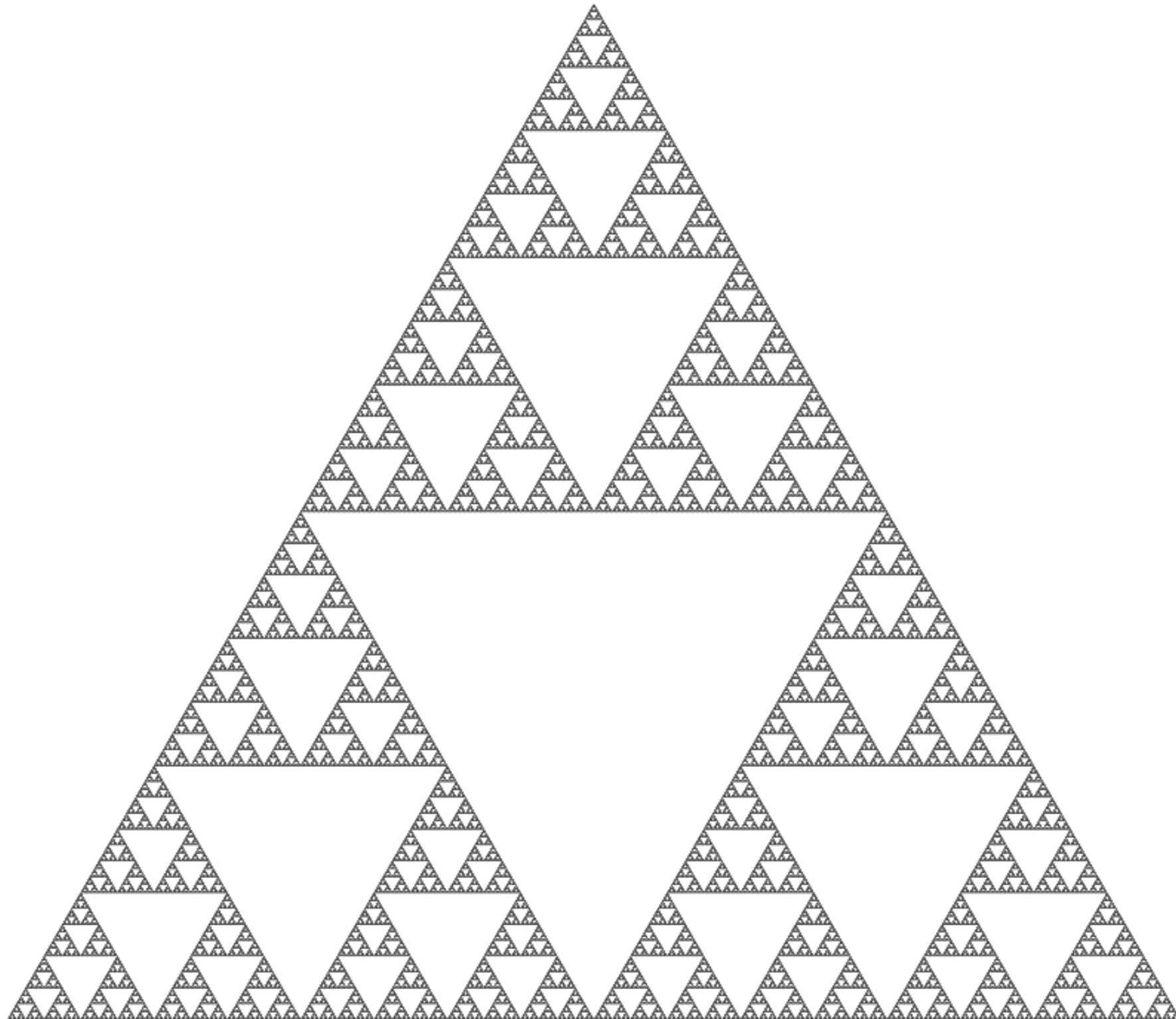
A recursive function usually has one or more ***base cases*** where a solution is produced without reoccurring. It also has ***recursive cases*** that result in the function calling itself with modified parameters.

Neglecting to implement a base case or to proceed towards a solution in the recursive cases can cause an infinite loop.

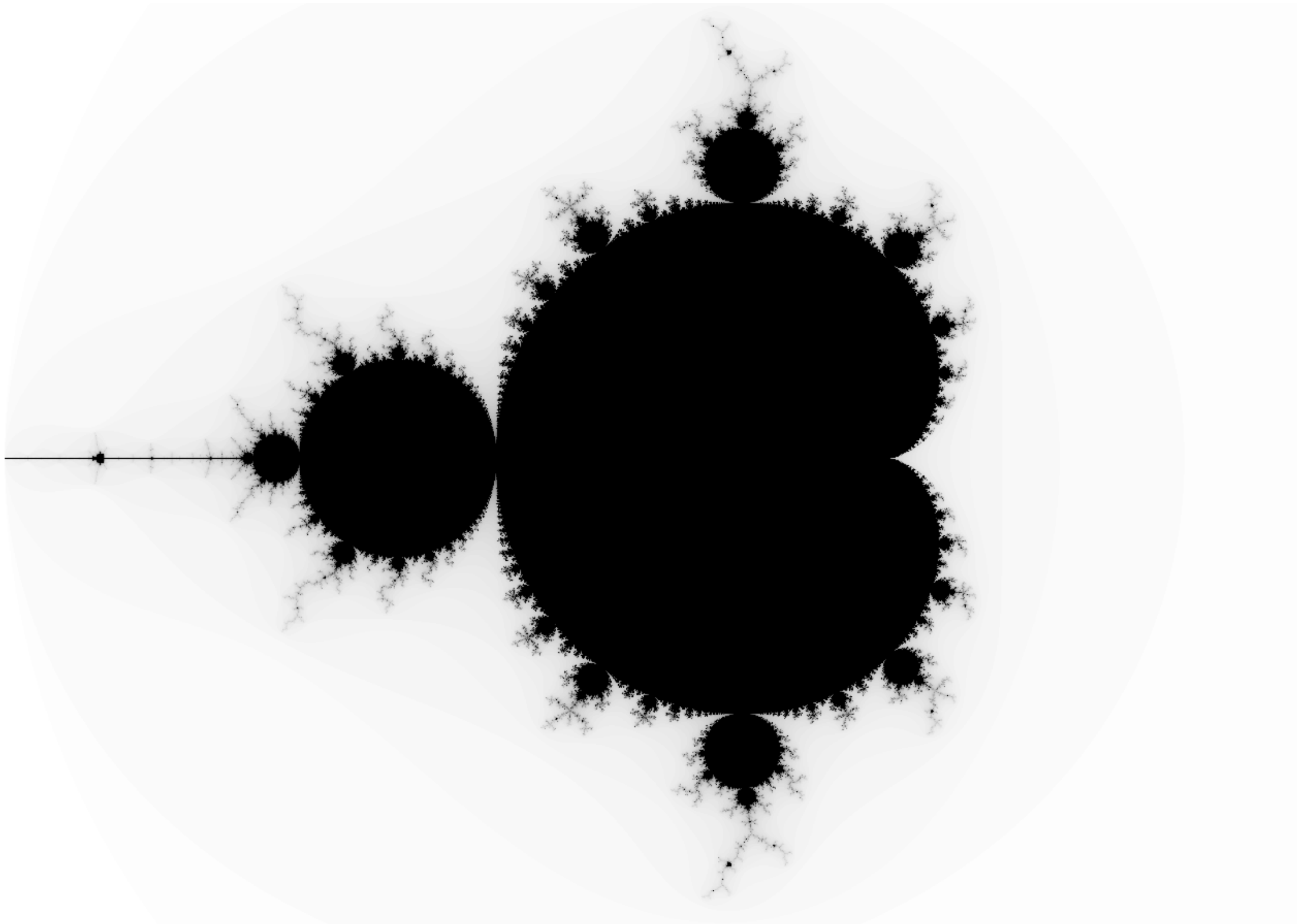
Cantor



Sierpinski



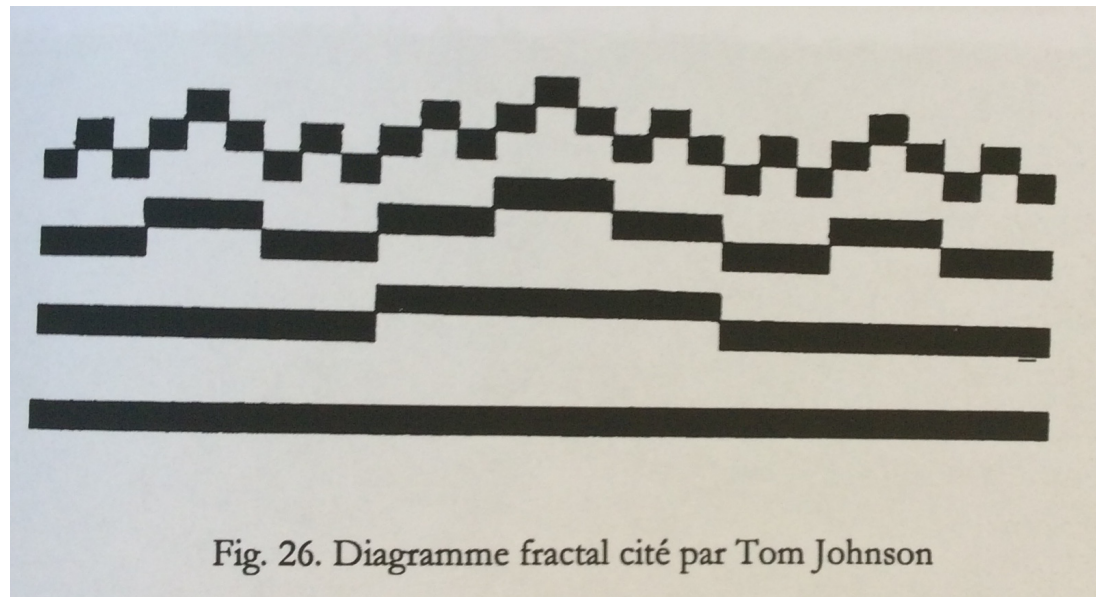
Mandelbrot



Fractals - Algorithm

```
Start with a single horizontal line segment.  
Repeat for a sufficiently large number of times {  
  Repeat over each line segment in the scene {  
    Find the midpoint of the line segment.  
    Displace the midpoint in Y by a random amount.  
    Reduce the range for random numbers.  
  }  
}
```

Thom Johnson



<http://ssm.editions75.com>

Fig. 26. Diagramme fractal cité par Tom Johnson

"Self-similar melodies are melodies constructed entirely by repeated applications of a single procedure."

"Self-similar music simply as music that somehow contains itself within itself and does so on at least three different levels of time."

On Cage learning from Schoenberg "I think the most important thing I learned from him was that the microcosm and the macrocosm should be related. Schoenberg talked often about that, and started me thinking in this direction."

Gary Lee Nelson

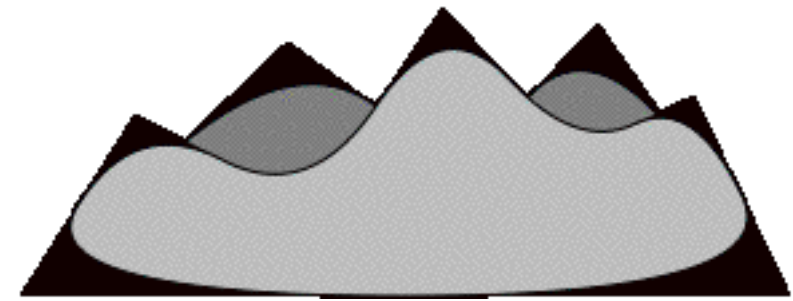
Faculty member at the Oberlin Conservatory of Music.

Attended Utrecht University's Institute of Sonology in 1964.

Composed ***Fractal Mountains (1988-89)*** using fractal concepts.

“Musical sequences can be generated by mapping the meeting points of each pair of lines onto time and pitch. Each vertex represents the attack point of a note. A second fractal function is used to translate dynamics into MIDI key velocity. The durations of notes are set by another fractal function that shapes texture by controlling the number of notes that are sounding at once. To return to our island analogy, we are determining whether the beach is covered with rocks, pebbles, or fine sand.”

Gary Lee Nelson



“The harmonic shape in "Fractal Mountains" comes from the tendency of the fractal model to be attracted to particular numbers and thus to particular pitches and MIDI channels. The pull of these "strange attractors" is one of the most interesting feature of fractals. In music they produce a sense of tension and release that is appealing physically as well as esthetically. Phrases begin with most notes in a single channel. As the phrase progresses, the notes fan out among the eight channels and the richness of the micro tonal palette is revealed. Near the end of each phrase the notes move toward a different channel and cadence in relative consonance. The center of gravity is constantly changing.”

<http://www.timara.oberlin.edu/gnelson/gnelson.htm>

Gary Lee Nelson - Fractal Mountains (1988)



1/f Noise

1/f noise

1/f noise refers to the phenomenon of the spectral density of a stochastic process.

1/f **fluctuations** are widely found in nature. During 80 years since the first observation by Johnson (1925), long-memory processes with long-term correlations have been observed in physics, technology, biology, astrophysics, geophysics, economics, psychology, language and even music.

1/f noise has '**self-similar**' characteristic where local details mirror the overall structure.

1/f noise

Research by **Voss** and **Clarke** (1978) found that various types of music and speech reflect 1/f fluctuations in pitch and loudness.

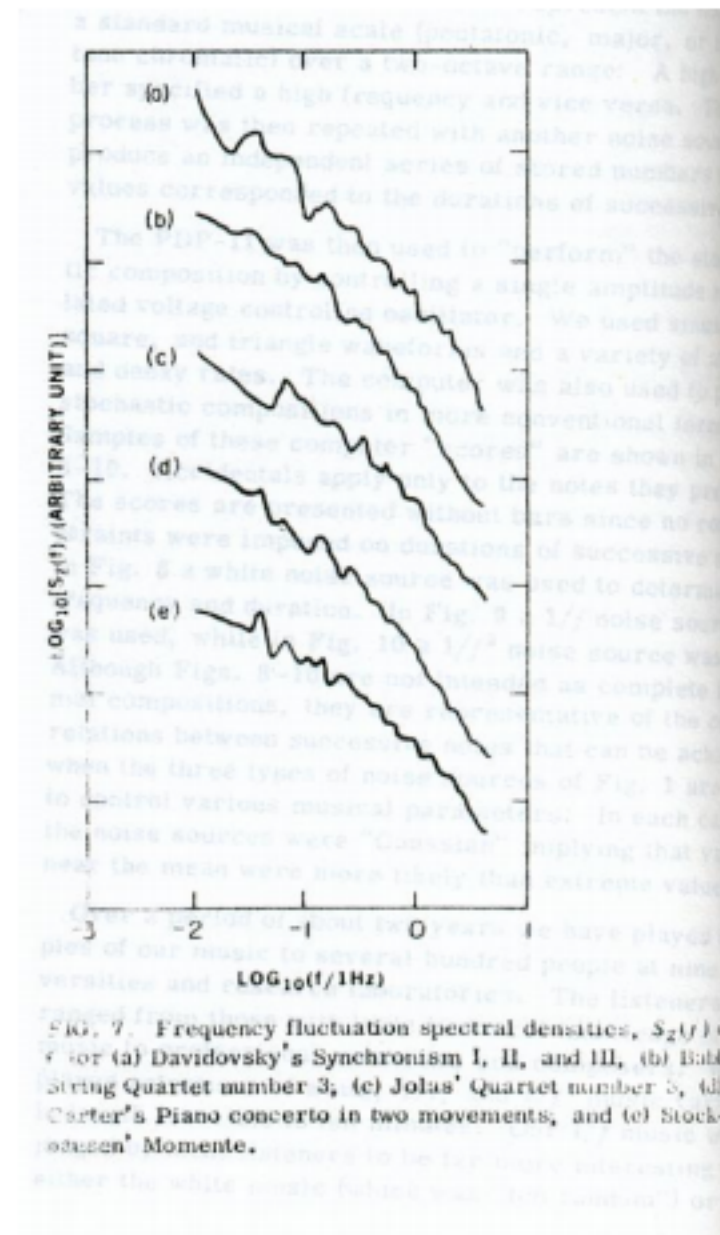
Voss thought that intelligent behavior should show 1/f spectral density.

Is quoted saying “the small voltages measurable across nerve membranes due to sodium and potassium flow” are 1/f noise and “1/f noise (perhaps that found in nerve membranes) may have an essential role in the creative process”

"1/f noise source would be a natural means of adding a large scale structure which is missing from most stochastic music since it is based on a white noise source which is uncorrelated in time, therefore structureless"

1/f noise

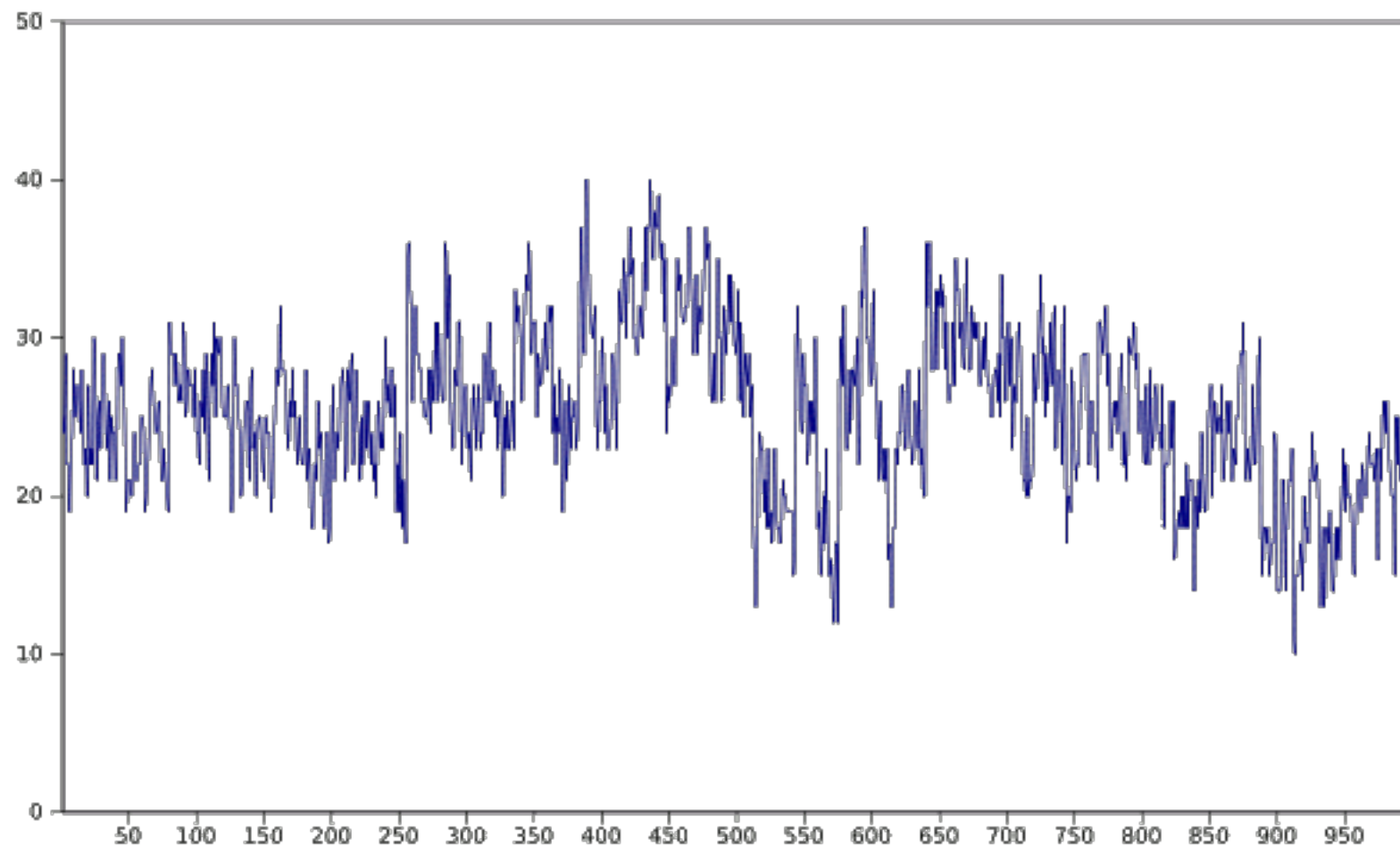
Davidovsky
Babbitt
Jolas
Carter
Stockhausen



Conclusion (Voss&Clarke):

Davidovsky the closest to 1/f while the others show decreased correlations for times longer than 10 seconds

1/f noise



Birkhoff's (1884-1944) theory of aesthetic value: *“for a work of art to be pleasing and interesting it should neither be too regular and predictable nor pack too many surprises.”*

1/f noise - Algorithm (Voss)

In to produce $2n$ values, one needs n dice runs.

For 8 values - $2^{\log(8)} - 3$ dices are needed.

One die changes every time, one every second time, one every fourth time, etc. See example below for 0-7.

$$6\ 3\ 4 = 13$$

$$6\ 3\ 5 = 14$$

$$6\ 4\ 1 = 11$$

$$6\ 4\ 3 = 13$$

$$3\ 1\ 6 = 10$$

$$3\ 1\ 5 = 9$$

$$3\ 2\ 2 = 7$$

$$3\ 2\ 4 = 9$$

```

(
  NF(\iop, {|freq=78, mul=1.0, add=0.0|
    var noise = LFNoise1.ar(0.001).range(freq, freq + (freq * 0.1));
    var osc = SinOsc.ar([noise, noise * 1.04, noise * 1.02, noise * 1.08],0,0.2);
    var out = DFm1.ar(osc,freq*4,SinOsc.kr(0.01).range(0.92,1.05),1,0,0.005,0.7);
    HPF.ar(out, 40)
  }).play;
)

(
  NF(\dsc, {|freq = 1080|
    HPF.ar(
      BBandStop.ar(Saw.ar(LFNoise1.ar([19,12]).range(freq,freq*2), 0.2).excess(
        SinOsc.ar( [freq + 6, freq + 4, freq + 2, freq + 8])),
        LFNoise1.ar([12,14,10]).range(100,900),
        SinOsc.ar(20).range(9,11)
      ), 80)
    ).play;
)

var <>pindex, <>cindex;

initialize {
  if(pindex.isNil, { pindex = 1000 });
  if(cindex.isNil, { cindex = 2000 });
}

clearProcessSlots {
  pindex = 1000;
  (this.pindex - 1000).do{|i| this[this.pindex+i] = nil; }
}

clearOrInit {|clear=true|
  if(clear == true, { this.clearProcessSlots() }, { this.initialize() });
}

transform {|process, index|
  if(index.isNil && pindex.isNil, {
    this.initialize();
  });

  pindex = pindex + 1;
  this[pindex] = \filter -> process;
}

control {|process, index|
  var i = index;

  if(i.isNil, {
    this.initialize();
    cindex = cindex + 1;
    i = cindex;
  });

  this[i] = \pset -> process;
}

(
  NF(\depfm, {|freqMin=5, freqMax=20, mul=20, add=80, rate=0.5, modFreq=2100, index=0.3, amp=0.2|
    var trig, seq, freq;
    trig = Dust.kr(rate);
    seq = Diwhite(freqMin, freqMax, inf).midicps;
    freq = Demand.kr(trig, 0, seq);
    HPF.ar(PMOsc.ar(LFCub.kr([freq, freq/2, freq/3, freq/4], 0, mul, add),
      LFNoise1.ar(0.3).range(modFreq,modFreq*2), index) * amp, 50)
  }).play;
)

```

Exercises

Exercises

1. Make a generative grammar for reading a buffer. Try to design carefully the grammar so that different parts of the buffer appear in varying ways.
2. L-Systems are known to be able to generate complex rhythmical patterns. Using the Nature Toolkits patterns, implement a rhythmic development of an L-System with a simple sound source. The system should influence the duration of events but always take a previous duration into account (not simply mapping a value to duration but mapping a value to a dynamic modifier of duration).
3. Implement a fractal process for playing back patterns. The process should be recursive and self-similar.
4. Implement the $1/f$ noise algorithm described on slide 34 in code.